Introduction

Hi Vikram, thank you for taking the time to speak with me today. My name is Sahaj Bhadja, and I recently completed my Master's in Computer Science from the University of Southern California.

I'm a full-stack software engineer with over two years of experience building secure and scalable applications using React, Next.js, TypeScript, Python, Node.js, REST APIs, and databases such as PostgreSQL and MongoDB.

Currently, at USC Dornsife, I work on full-stack applications with React and Next.js on the frontend and Python services on the backend. I have also integrated generative and agentic AI capabilities into enterprise workflows while following secure development, code-review, testing, and CI/CD practices.

Previously, at Aasma Technology Solutions, I developed secure full-stack features using React, TypeScript, Flask, PostgreSQL, and MongoDB. I also worked with microservices, automated testing, Git-based workflows, and continuous deployment.

What particularly interests me about this Deloitte opportunity is the combination of full-stack engineering, cybersecurity, and client-facing problem-solving. I enjoy taking an ambiguous business requirement, converting it into a clear technical solution, and communicating the reasoning and trade-offs behind my approach.

Behavioral Interview Story Bank

1. USC Dornsife — Software Engineer

Detailed description

At USC Dornsife, I work as a software engineer building and improving full-stack applications used in enterprise-style workflows. My primary frontend stack includes Next.js, React, TypeScript, HTML, and CSS, while the backend consists of Python-based services deployed on AWS.

My responsibility extends beyond implementing individual frontend components. I work across the full application flow: understanding a business requirement, translating it into technical tasks, designing the user interaction, connecting the frontend to backend services, validating requests, handling errors, testing the feature, and supporting its deployment.

A major part of my work has involved integrating generative AI and agentic AI capabilities into existing workflows. This requires more than simply calling a language-model API. I have to consider how data is passed into the model, how responses are validated, how failures are handled, and how the AI functionality fits into the larger application. Because AI responses can be unpredictable, I focus on providing clear system instructions, validating outputs, handling timeouts or incomplete responses, and preventing sensitive information from being exposed unnecessarily.

I also follow secure software-development practices through code reviews, testing, Git-based collaboration, and CI/CD pipelines. This experience has taught me how to balance speed of development with maintainability, security, and production reliability.

Interview-ready answer

“At USC Dornsife, I work as a full-stack software engineer building applications with Next.js, React, TypeScript, and Python services deployed on AWS. I usually work across the complete feature lifecycle, starting with understanding the requirement and then implementing the frontend, connecting it to backend services, handling validation and error cases, testing the solution, and supporting deployment.

One especially valuable part of the role has been integrating generative and agentic AI into enterprise workflows. The challenge with AI features is that the response is not always deterministic, so I cannot treat the model like a traditional function that always returns a predictable result. I have to think about prompt structure, output validation, failure handling, security, and the user experience when the model is unavailable or produces an incomplete response.

This role has strengthened both my technical skills and my ability to work through ambiguous requirements. It has also taught me to build features that are not only functional, but maintainable, secure, and appropriate for real users.”

Strong behavioral questions for this experience

Use this experience for:

- Tell me about a time you worked with ambiguous requirements.
- Tell me about a technically challenging project.
- Tell me about a time you learned a new technology.
- Tell me about a time you considered security during development.
- Tell me about a time you built an AI-enabled application.
- Tell me about a time you owned a feature end to end.
- How do you ensure code quality?

Strong challenge angle

A strong challenge to discuss is integrating nondeterministic AI behavior into a predictable user-facing application.

You can explain that your solution involved:

- Clearly defining the expected model output.
- Validating the returned response.
- Handling timeouts and malformed responses.
- Providing fallbacks or clear error messages.
- Separating AI logic from core business logic.
- Testing normal and failure cases.
- Avoiding unnecessary exposure of sensitive information.

---

2. Grad8 — Software Engineer Intern

Detailed description

At Grad8, I worked as a software engineering intern building and consuming production REST APIs and microservices using Node.js and Python. My work involved connecting different parts of the application so that frontend or internal services could exchange data reliably through clearly defined API contracts.

I worked with containerized services using Docker, which helped ensure that the application behaved consistently across local development, testing, and deployment environments. Rather than relying on each developer to manually configure every dependency, the service and its runtime requirements could be packaged together.

I worked in an Agile and Scrum environment, where tasks were divided into sprints and discussed through planning sessions, stand-ups, reviews, and feedback cycles. This required me to communicate progress clearly, identify blockers early, and adjust my implementation when requirements changed.

One of the most important parts of the role was working with cross-functional stakeholders. A stakeholder might describe a business need rather than a technical specification, so I had to ask clarifying questions and translate that need into API behavior, validation rules, data structures, and user-facing functionality.

This experience taught me that successful software engineering is not only about writing correct code. It is also about understanding why the feature is needed, agreeing on the expected behavior, communicating trade-offs, and delivering something that solves the actual business problem.

Interview-ready answer

“At Grad8, I worked on production REST APIs and microservices using Node.js and Python. I helped implement backend functionality, integrate services, and package applications with Docker so that they could run consistently across environments.

The role was highly collaborative because I often had to translate business requirements from cross-functional stakeholders into technical functionality. I learned to avoid immediately jumping into implementation. Instead, I would first clarify the expected inputs, outputs, edge cases, and success criteria. I would then break the work into smaller tasks and keep the team informed during the sprint.

This experience improved my ability to work in an Agile environment and taught me that attention to detail is especially important when building APIs. Even a small misunderstanding in an API contract can affect multiple services or users, so I focused on clear interfaces, validation, testing, and communication.”

Strong behavioral questions for this experience

Use this experience for:

- Tell me about a time you worked with cross-functional stakeholders.
- Tell me about a time requirements changed.
- Tell me about a time you worked in an Agile team.
- Tell me about a time you had to pay close attention to detail.
- Tell me about a time you worked under a deadline.
- Tell me about a time you improved collaboration.
- Tell me about a time you received feedback.

Strong challenge angle

A strong challenge is converting a high-level business request into a precise technical API contract.

Your explanation can include:

- Asking who would consume the API.
- Clarifying required and optional fields.
- Defining validation rules.
- Agreeing on status codes and error responses.
- Checking compatibility with existing consumers.
- Testing both successful and unsuccessful requests.
- Documenting the API behavior for other engineers.

---

3. USC AD Labs — Machine Learning Research Assistant

Detailed description

At USC AD Labs, I worked as a machine learning research assistant developing and deploying AI solutions on Google Cloud Platform. My responsibilities included creating data pipelines that supported model-driven security analytics functionality.

A machine-learning feature depends on more than the model itself. Data has to be collected, transformed, validated, and passed to the model in the correct format. The output then has to be stored, interpreted, or presented to another service or user. I worked on making this flow observable so that the team could understand where a failure occurred and whether the pipeline was behaving correctly.

Observability was important because AI and data pipelines can fail silently. For example, a pipeline may continue running even though the input format has changed or the quality of the data has degraded. Logging, validation, and monitoring make it easier to identify these issues before they affect the final result.

I also contributed automated tests for model-driven security analytics features. These tests helped verify that the surrounding software behaved consistently even when the model or data introduced complexity.

Another important responsibility was contributing to technical analysis and solution-design documentation. I had to explain technical concepts to both engineers and nontechnical partners. This taught me to adjust the depth of my explanation depending on the audience while preserving the important technical reasoning.

Interview-ready answer

“At USC AD Labs, I worked on AI solutions deployed on GCP, with a particular focus on observable data pipelines and automated testing for model-driven security analytics features.

One lesson I learned was that deploying an AI solution involves much more than training or calling a model. The data pipeline, validation, logging, testing, and system integration are equally important. If an input changes or a processing step fails, the team needs enough visibility to identify the issue quickly.

I also contributed to technical analysis and solution-design documentation. This required me to communicate the same system differently depending on the audience. With engineers, I could discuss the pipeline, interfaces, and testing strategy in depth. With nontechnical partners, I focused more on the purpose of the system, the information it produced, and the limitations they needed to understand.

That experience helped me combine technical implementation with clear communication, which I believe is especially valuable in consulting and client-facing engineering.”

Strong behavioral questions for this experience

Use this experience for:

- Tell me about a time you explained something technical to a nontechnical person.
- Tell me about a time you improved observability.
- Tell me about a time you worked on an AI or machine-learning system.
- Tell me about a time testing prevented a problem.
- Tell me about a time you worked with incomplete or changing data.
- Tell me about a research problem you solved.
- How do you communicate technical limitations?

Strong challenge angle

A strong challenge is diagnosing problems across a multi-stage AI data pipeline.

A good structure is:

- Identify where data enters the system.
- Validate data at every major stage.
- Add structured logs and meaningful error messages.
- Separate model errors from infrastructure errors.
- Create tests with representative and invalid inputs.
- Document model and system limitations.
- Communicate findings without overstating confidence.

---

4. Aasma Technology Solutions — Software Developer Engineer, SecureSync

Detailed description

At Aasma Technology Solutions, I worked as a software developer engineer on SecureSync, where I built secure full-stack features using React, TypeScript, Python with Flask, PostgreSQL, and MongoDB.

On the frontend, I worked with reusable React components rather than creating isolated UI elements for every new feature. A reusable component architecture improves consistency and makes future changes easier because common behavior and styling are maintained in one place.

On the backend, I worked with Flask services and database integrations. PostgreSQL was useful for structured relational data and transactions, while MongoDB supported use cases

where a more flexible document structure was appropriate. Working with both taught me to select a database based on the structure, relationships, consistency requirements, and query patterns of the data.

Security was an important part of the application. I implemented or followed web-security controls around data validation, access, API handling, and safe communication between system components. This helped me understand that security should be included during design and development rather than added only after the application is completed.

I also maintained production serverless and microservice integrations. When services depend on one another, changes must be introduced carefully because an update to one service can break another. I used automated testing, Git workflows, and continuous deployment practices to make releases safer and more repeatable.

This experience gave me strong exposure to production software engineering because I had to consider not only implementation, but also integration, testing, security, maintainability, and deployment.

Interview-ready answer

“At Aasma Technology Solutions, I worked as a software developer engineer on SecureSync. I developed full-stack functionality using React and TypeScript on the frontend, Flask on the backend, and PostgreSQL and MongoDB for data storage.

I worked on reusable frontend components, API integrations, backend business logic, and database operations. Security was also an important consideration, so I paid attention to validation, access controls, secure API handling, and avoiding unnecessary exposure of sensitive data.

Another important part of my work was maintaining serverless and microservice integrations. Because services were connected, I had to consider the downstream impact of every change. I used automated testing, code reviews, Git workflows, and continuous deployment practices to reduce the chance of introducing regressions.

This role gave me experience building production applications end to end and showed me why maintainability and security are just as important as delivering the initial feature.”

Strong behavioral questions for this experience

Use this experience for:

- Tell me about a time you built a secure application.
- Tell me about a time you owned a full-stack feature.
- Tell me about a difficult integration.
- Tell me about a production issue.

- Tell me about a time you prevented a regression.
- Tell me about a technical trade-off.
- Tell me about a time you improved code maintainability.
- Tell me about a time you worked independently.

Strong security angle

When discussing secure development, mention:

- Validating user input on the backend.
- Enforcing authorization at the API layer.
- Using parameterized queries.
- Avoiding sensitive information in logs.
- Keeping secrets outside source code.
- Applying least-privilege access.
- Returning safe error messages.
- Testing unauthorized and invalid requests.

Strong database trade-off angle

You can explain:

“I would use PostgreSQL when the data has clear relationships, requires transactional consistency, or needs structured joins. I would consider MongoDB when the data structure is more flexible or document-oriented. The decision should be based on access patterns and consistency requirements rather than choosing a database only because it is familiar.”

---

5. CyberTech Solutions — Full-Stack Developer Intern

Detailed description

At CyberTech Solutions, I worked as a full-stack developer intern building cybersecurity dashboard applications. The frontend was built with React and JavaScript, while Python services handled backend functionality and integration with threat-data APIs.

The dashboard’s purpose was to take technical cybersecurity information and present it in a format that users could understand and act on. Threat-data APIs may provide detailed or inconsistent data, so the application needed to transform that information into clear UI elements, statuses, tables, or summaries.

I supported secure and responsive user interfaces, meaning the application needed to work across screen sizes while handling cybersecurity data carefully. I also applied clean coding

practices and object-oriented design principles so that the code remained understandable and maintainable as functionality expanded.

Because this was an internship in a fast-paced environment, I had to learn the existing codebase quickly, ask focused questions, and contribute without requiring constant supervision. I worked both independently and with other engineers, which improved my ability to communicate blockers and receive technical feedback.

This experience is particularly relevant to Deloitte Cyber because it introduced me to the intersection of full-stack engineering, cybersecurity information, API integration, and user-facing decision support.

Interview-ready answer

“At CyberTech Solutions, I worked on cybersecurity dashboard applications using React, JavaScript, and Python. I integrated threat-data APIs and helped transform technical security information into a responsive interface that users could understand more easily.

The interesting challenge was that raw cybersecurity information can be complex and difficult to interpret. The application needed to present the important information clearly without hiding necessary context. I therefore had to think about both the technical integration and the user experience.

I also focused on clean code and object-oriented design so that the system would be easier to maintain as additional threat sources or dashboard features were added. Since it was an internship, I had to learn quickly, understand an existing codebase, and balance independent work with asking the right questions.

That experience was one of the reasons I became interested in combining cybersecurity with full-stack engineering.”

Strong behavioral questions for this experience

Use this experience for:

- Why are you interested in cybersecurity?
- Tell me about your first relevant technical experience.
- Tell me about a time you learned a codebase quickly.
- Tell me about a time you worked independently.
- Tell me about a time you presented complex information clearly.
- Tell me about an API integration.
- Tell me about a time you received mentorship or feedback.

Projects

6. CyberGuard AI

Detailed description

CyberGuard AI is an enterprise-style cybersecurity application built with Next.js, Python, AWS, and generative AI. The purpose of the project is to use AI to support the analysis of cybersecurity threats while providing the functionality through a secure full-stack application.

The frontend provides the user interface and sends requests to Python backend services. The backend is responsible for validating requests, communicating with the AI or threat-analysis functionality, handling errors, and returning a structured result to the frontend.

The generative-AI component helps interpret or summarize threat-related information. However, because AI output can be incomplete or incorrect, the application should treat it as analytical assistance rather than unquestionable truth. The system must clearly distinguish between source data and generated interpretation.

Security is especially important because the application handles cybersecurity-related information. Secure API integration, backend validation, controlled access, secrets management, safe logging, and appropriate cloud permissions are all important design considerations.

AWS provides the deployment environment. When discussing the project, focus on the overall application architecture, data flow, security controls, and how the system could be made production-ready.

Interview-ready answer

“CyberGuard AI is a full-stack cybersecurity application I built using Next.js, Python, AWS, and generative AI. The goal was to explore how AI could support threat analysis while still being integrated into a secure and maintainable application.

The frontend collects the relevant user input and communicates with the Python backend through an API. The backend validates the request, invokes the analysis functionality, processes the response, and returns structured information to the frontend.

One of the main design considerations was that generative-AI output should not automatically be treated as verified security intelligence. The application needs to preserve the original threat information, clearly identify AI-generated interpretation, and handle cases where the output is incomplete or unavailable.

The project helped me think beyond model integration and consider API security, cloud deployment, error handling, observability, and how an AI-enabled cybersecurity tool could be made suitable for enterprise use.”

Best behavioral questions

- Tell me about a project you are proud of.
- Tell me about a project relevant to cybersecurity.
- Tell me about a time you combined multiple technologies.
- How do you handle security in an AI application?
- Tell me about a technical design decision.
- Tell me about a project you built independently.
- How would you take a prototype to production?

Production improvements to mention

With more time, you could discuss adding:

- Role-based access control.
- Audit logging.
- Rate limiting.
- Secure secrets management.
- Human review for high-risk conclusions.
- Stronger output validation.
- Monitoring and alerting.
- Automated security testing.
- Caching and asynchronous processing.
- Evaluation of AI output quality.

---

7. Big-Brain Live

Detailed description

Big-Brain Live is a generative-AI application built using React, Python, GCP, and large-language-model integration. The project focuses on maintainable full-stack architecture rather than treating the AI model as an isolated demo.

The React frontend provides the user interaction, while Python backend services manage requests to the language model and other application logic. Keeping the model interaction on the backend prevents credentials and sensitive implementation details from being exposed in the browser.

The application includes automated testing and production observability. Testing verifies that the application behaves correctly for expected inputs, invalid requests, and service failures. Observability helps track request flow, failures, response time, and the behavior of external model dependencies.

A major architectural consideration is separating different concerns. The frontend should focus on presentation, the API layer should manage validation and request handling, the service layer should manage model interactions, and infrastructure should manage deployment and monitoring.

This separation makes the application easier to test, debug, maintain, and extend if the model provider or product requirements change.

Interview-ready answer

“Big-Brain Live is a generative-AI application I developed with React, Python, GCP, and LLM integration. My goal was to build it as a maintainable full-stack system rather than only demonstrating that I could call a model API.

I separated the frontend, backend API, model-service logic, and infrastructure concerns. This made it easier to test each part independently and reduced the impact of changing one component.

I also included automated testing and observability because external AI services can introduce latency, errors, and unpredictable output. The application needs to capture enough information to diagnose failures while avoiding logging sensitive user data.

The project taught me that production AI engineering is largely a software-engineering and systems problem. Model capability is important, but reliability, security, testing, monitoring, and user experience determine whether the application is actually useful.”

Best behavioral questions

- Tell me about a time you designed a maintainable system.
- Tell me about an AI project.
- Tell me about a time you improved observability.
- Tell me about a difficult external-service integration.
- Tell me about a time you planned for failure.
- Tell me about a technical trade-off.
- How do you test an AI-enabled application?

---

8. FinNexus Live

Detailed description

FinNexus Live is a secure and scalable full-stack platform built with React, Node.js, PostgreSQL, MongoDB, REST APIs, and a CI/CD delivery pipeline.

The frontend communicates with Node.js backend services through REST APIs. The backend contains the application's business logic and interacts with both relational and NoSQL data stores.

Using PostgreSQL and MongoDB in the same project demonstrates a polyglot-persistence approach. PostgreSQL can support structured and relational data that benefits from transactions and clear schemas. MongoDB can support flexible document-oriented information. The important consideration is assigning each type of data to the database that best fits its access and consistency requirements.

The CI/CD pipeline helps automate the process of validating and delivering changes. A typical flow includes checking out the code, installing dependencies, running tests, building the application, and deploying only after required checks pass.

Security and scalability should be addressed across the application rather than in one isolated layer. This includes validation, authorization, database security, safe API design, configuration management, monitoring, and the ability to scale backend services independently.

Interview-ready answer

"FinNexus Live is a full-stack platform I developed using React, Node.js, PostgreSQL, MongoDB, REST APIs, and CI/CD.

One of the main technical decisions was using both relational and NoSQL databases. I treated that as a data-modeling decision rather than using both unnecessarily. Structured data with relationships and transactional requirements was better suited to PostgreSQL, while flexible document-oriented data could be handled through MongoDB.

The frontend communicated with the backend through REST APIs, and the backend encapsulated the business logic and database access. I also incorporated a CI/CD pipeline so that testing and deployment could be more consistent and repeatable.

This project strengthened my understanding of full-stack architecture, database trade-offs, API design, and the operational work required to deliver an application safely."

Best behavioral questions

- Tell me about a technical trade-off.
- Tell me about a scalable system you designed.
- Tell me about your database experience.

- Tell me about a CI/CD pipeline.
- Tell me about a secure full-stack application.
- Tell me about a project involving API design.
- Why did you use both SQL and NoSQL?

Behavioral Question-to-Story Map

“Tell me about a project you are proud of.”

Best story: **CyberGuard AI**

Why: It directly combines full-stack engineering, cybersecurity, GenAI, AWS, secure APIs, and enterprise design.

“Tell me about a technically difficult problem.”

Best story: **USC Dornsife**

Angle: Integrating nondeterministic AI behavior into a predictable enterprise workflow.

“Tell me about a time requirements were unclear.”

Best story: **Grad8**

Angle: Translating a business request from stakeholders into an API contract and technical implementation.

“Tell me about a time you considered security.”

Best story: **Aasma SecureSync**

Angle: Applying backend validation, API-layer authorization, secure data handling, and testing rather than relying only on frontend controls.

“Tell me about a time you explained something technical.”

Best story: **USC AD Labs**

Angle: Explaining an AI and data-pipeline design differently to engineers and nontechnical stakeholders.

“Tell me about a time you learned quickly.”

Best story: **CyberTech Solutions**

Angle: Learning an existing cybersecurity dashboard codebase and contributing in a fast-paced environment.

“Tell me about a time you worked across teams.”

Best story: **Grad8**

Angle: Collaborating with cross-functional stakeholders during Agile sprints.

“Tell me about a time you improved maintainability.”

Best story: **Aasma or Big-Brain Live**

Angle: Reusable React components, separation of concerns, automated testing, and structured service boundaries.

“Tell me about a failure or mistake.”

Your resume does not specify a real failure, so do not invent one. Choose a genuine situation where:

- You initially misunderstood a requirement.
- A test exposed an edge case.
- An integration failed because of an incorrect assumption.
- You underestimated the effort required.
- You received feedback that caused you to redesign something.

Structure the answer around taking ownership, correcting the issue, and changing your process.

“Tell me about a conflict.”

Your resume does not identify a specific conflict. Use a real example involving a technical disagreement rather than inventing interpersonal drama.

A strong structure is:

- You and another person preferred different solutions.
- You clarified the shared objective.
- You compared both solutions using requirements and trade-offs.
- You suggested a small prototype, test, or data-based evaluation.
- The team selected an approach.
- You supported the final decision even if it was not entirely your original idea.

“Why Deloitte Cyber?”

“My experience sits at the intersection of full-stack development, cybersecurity, cloud systems, and AI. I have built applications using React, Next.js, TypeScript, Python, Node.js, relational and NoSQL databases, and cloud platforms. I have also worked on cybersecurity dashboards, security analytics features, and secure AI-enabled applications.

What attracts me to Deloitte Cyber is the opportunity to apply those skills to complex client problems. I am particularly interested in work where I can understand a client’s security or business requirement, design a practical technical solution, build it securely, and communicate the reasoning clearly to both technical and nontechnical stakeholders.”

STAR Answer Framework

For every answer, use this structure:

Situation: Give only enough context to understand the problem.

Task: Clearly state your responsibility.

Action: Spend most of the answer here. Explain what you personally analyzed, decided, built, communicated, or changed.

Result: State the measurable result when you have one. Otherwise, explain what improved, what was delivered, or what you learned without inventing a number.

A strong behavioral answer should generally last between 90 seconds and two minutes.